NOTE Some functions that follow the specification do not implement the abstract factory pattern in the traditional sense. For example, instead of using classes for the factory, they use regular functions that hold no state. It's important to understand that it's an elegant way of introducing new variants of models without breaking existing client code.

Both `generateText` and `streamText` functions act as abstract factories (listing 3.10). They accept a configuration object that satisfies their interface requirements, which defines the contract for creating instances of language models.

The language model factory interface declares the `generateText` and `streamText` methods. Each provider implementation, such as OpenAI and Anthropic, needs to implement this interface and provide a way to satisfy these methods.

The OpenAI and Anthropic classes are concrete factories that implement the language model factory interface. Their constructors take a model name as a parameter and return an instance of the corresponding language model (`OpenAIModel` or `AnthropicModel`).

The language model specification model interface defines the contract for generating and streaming text based on a prompt. The key point here is that each provider's language model needs to implement the language model specification model interface to define how to generate or stream text based on a prompt, and that pattern decouples the client code from the concrete language model implementations, allowing for easy integration of new providers.

Let's recap:

- *Abstract factories*—The `generateText` and `streamText` functions act as abstract factories, providing a high-level interface for generating or streaming text using language models.
- *Concrete factories*—Provider-specific modules like `@ai-sdk/openai` implement concrete factories that create instances of specific language model implementations.
- *Abstract products*—The language model specification model interface defines the contract for generating and streaming text based on prompts.
- *Concrete products*—Implementations like `OpenAIModel` and `AnthropicModel` are concrete products that implement the language model specification model interface for their respective providers.
- *Decoupling*—The abstract factory pattern decouples the client code from the concrete language model implementations, allowing easy integration of new providers without modifying the existing code.

Now that you have a rough idea of what the language model specification is, we take a brief look at the supported providers.

SUPPORTED PROVIDERS

The Vercel AI SDK supports several popular AI providers out of the box. Table 3.4 lists the provider together with the npm package in which it resides.